CADT IDT

W9 PRACTICE

QUIZ APP

Important

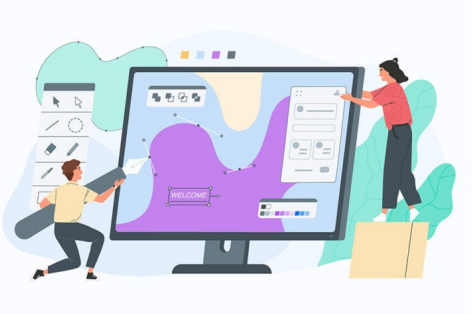
- ✓ The **reflection part** will be done in **teams of 2** (*designing*) and 4 (*sharing*)
- ✓ The **coding part** needs to be submitted **individually**

Learning objectives

- ✓ Handle navigation between multiple screens – Using a state (not router for now…)
- ✓ Pass data between screens
- ✓ Separate UI logic from business logic: using a model folder
- ✓ Reflect on the best approaches (data, states, widgets) to maintain a clean architecture

How to submit?

- ✔️ **Push** your final code on **your GitHub repository**
- ✔️ Then **attach the GitHub path** to the MS Team assignment and **turn it in**



Functional Requirements

For this practice (W9)

- ✓ The player can **start the quiz** and **answer each question** one by one
- ✓ Only single choice questions
- ✓ Once finished, the app shows the **score and the questions results**

For Bonus

- ✓ The history of the previous scores can be reviewed
- ✓ The **quiz questions** and **player submission** are persisted in JSON file

For next practice (W10)

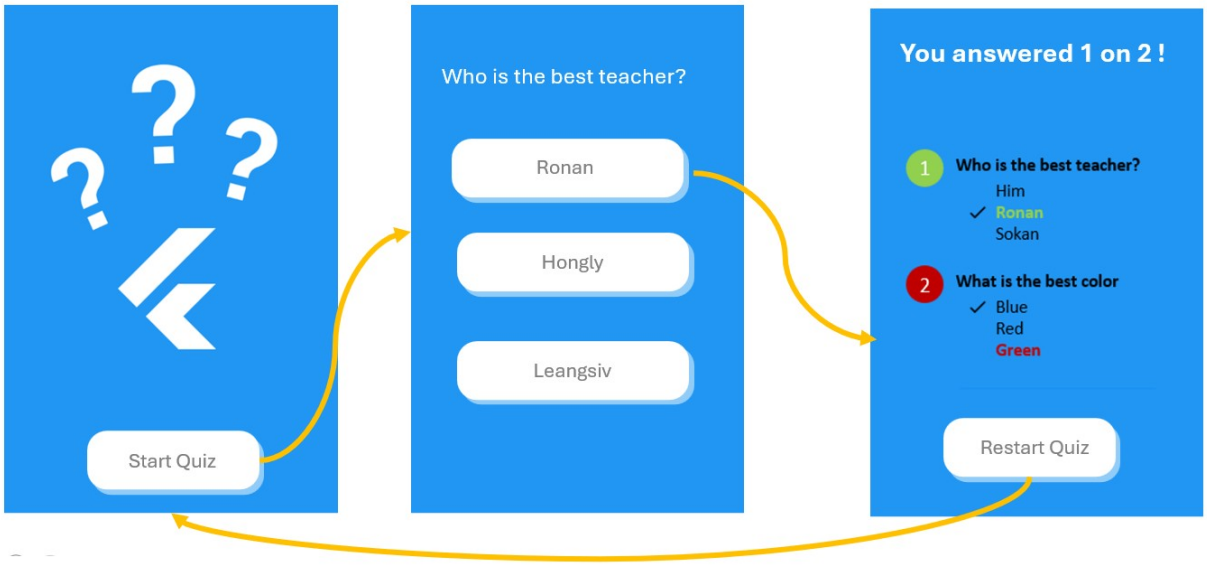
- ✓ The player **enters his/her name** before starting
- ✓ It's possible to **edit the quiz questions**

Non-Functional Requirements

- ✓ The application must **implement the provided user flow and mockups**

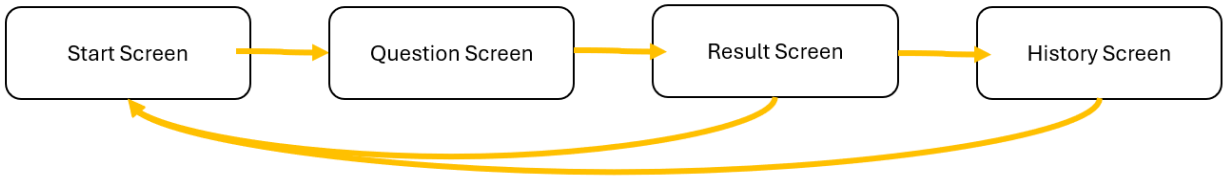
User Flow

For this practice, the following **user flow**/mockup are required:



BONUS

To include the **history of the previous scores**, the **user flow** can evolve as follows:



Layer structure

The application is structured around 3 layers: DATA > DOMAIN > UI

data	Repositories to load **domain objects** from **data sources**
model	Contain the **domain classes**
ui/screen	Screen widgets and sub-screen widgets
ui/widgets	Re usable widgets (button, inputs…)

Here is an **example** of project structure *(just an example, not the correct one)*

```
lib/
 ├─ data/
 │    └─ repositories/
 │         └─ quiz_json_repository.dart
 │                   └─ quiz_mock_repository.dart
 ├─ models/
 │    └─ quiz.dart
 │
 └─ ui/
     ├─ screens/
     │    └─ welcome_screen.dart
     │    └─ question_screen.dart
     └─ widgets/
          └─ app_button.dart
               └─ app_button.dart

          main.dart
```

Layer interaction

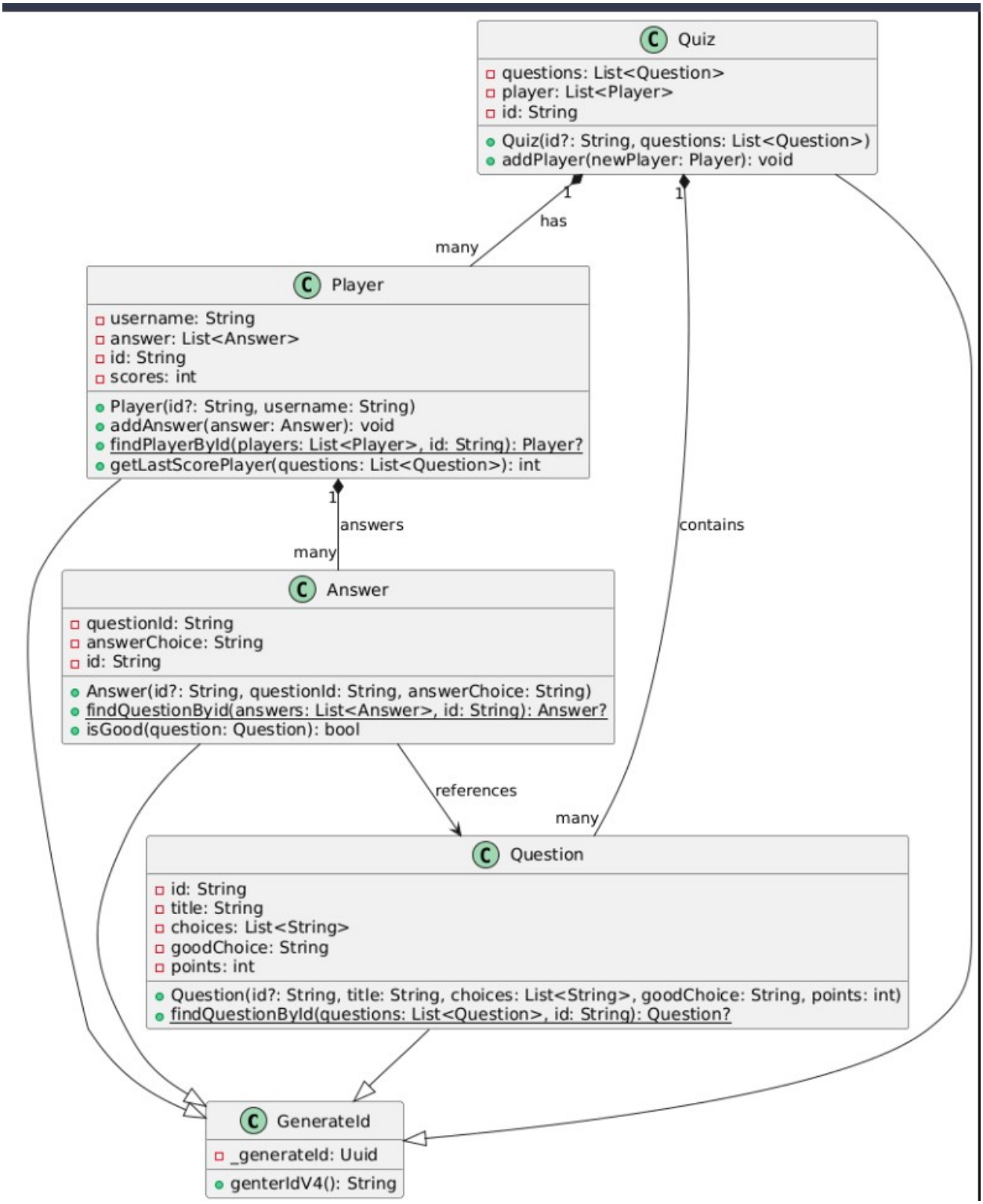
1. The **main** loads the **quiz data** (*from mock data or from a Json file)*
2. The **main** create the **quiz screen**, passing the quiz data as parameter

PART 1 – REFLECTIONS

MODEL

To handle the functional requirements for this practice, and be ready for the next practice, how are you going to structure your model?

Q1 – Drop below the **UML diagram** **of your model**



Q2 – Where do you **keep player submission,** so that you can display the last screen?

I will keep the player submission in a **list stored in the main quiz state**, so the data can be passed to the final result screen. Every time the user selects an answer for a question, I will add it to the list. Then, on the last screen, I can use that stored data to display whether each answer is correct or wrong.

UI – Screens

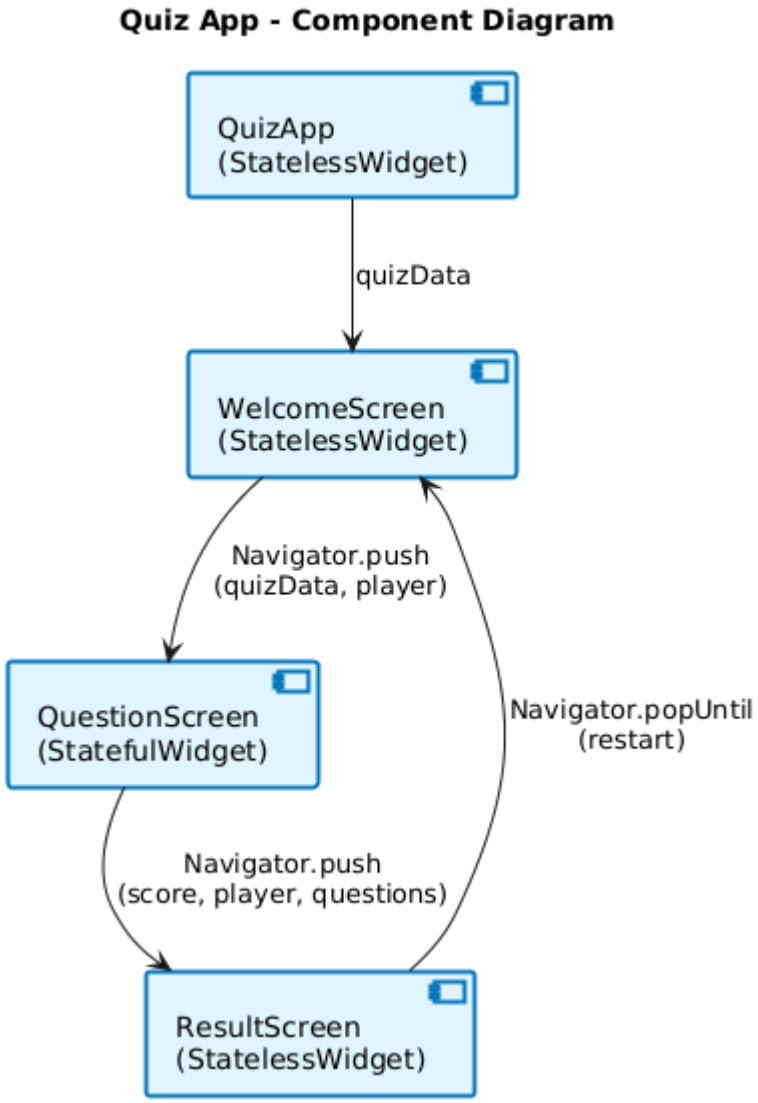
We have 3 screens (start, question and result)

Q3 – Identify for **each screens** their properties

SCREEN	TYPE (SL / SF)	PARAMETERS	STATES
(WelcomeScreen)	SL	quizData	no
QuestionScreen	SF	quizData, player	currentQuestionIndex
ResultScreen	SL	score, totalQuestions, player, questions	no

Q4 – Draw the **COMPONENT DIAGRAM** of the application

{HERE}



Q5 – Where and How do you **manage the navigation** to the **next questions** and to the **last result screen**?

{HERE}
Navigation is managed in QuestionScreen using Navigator.push():

- After answering each question, currentQuestionIndex state increments
- When all questions are answered, Navigator.push() navigates to ResultScreen
- From ResultScreen, Navigator.popUntil((route) => route.isFirst) returns to WelcomeScreen

UI – Reusable widget

List down the widget you are **planning to re-use** on different screens (button, card..)

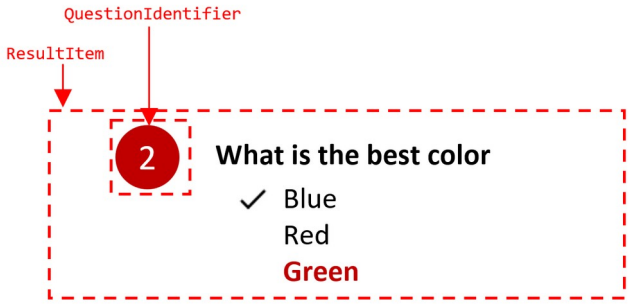
WIDGET	TYPE (SL / SF)	PARAMETERS	STATES
AppButton	SL	label, onTap, icon	no
QuestionIdentifier	SL	questionNumber	no
DisplayPoint	SL		no
Title	SL		no

PART 2 – IMPLEMENTATION

- ✓ The **coding part** needs to be submitted **individually**

HINTS

- ✓ Tip: you can divide each screen into many **stateless screen-widgets**, for example:



This widget takes as parameter a question and a player choice and handle the color computation, the choices highlighting etc..